Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 3

Cài đặt và thực thi mục 1:

- Hàm find_minimum_edit_distance phần tìm minimun edit distance:

```python
def find_minimum_edit_distance(source_string, target_string):
    m = len(source_string)
    n = len(target_string)

    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1 , m + 1):
        dp[i][0] = i

    for j in range(1 , n + 1):
        dp[0][j] = j

    operations_performed = []

    for i in range(1 , m + 1):
        for j in range(1 , n + 1):
            if source_string[i - 1] == target_string[j - 1]:
                a = 0
            else:
                a = 2
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + a)

```

Sửa lỗi kích thước mảng dp thành [[0] * (n + 1) for _ in range(m + 1)]

- Phần tìm các phép cụ thể:

```python
    i = len(source_string)
    j = len(target_string)

    while(i != 0 and j != 0):
        if source_string[i - 1] == target_string[j - 1]:
            i -= 1
            j -= 1
        else:
            if dp[i][j] == dp[i - 1][j - 1] + 2:
                operations_performed.append(('REPLACEMENT', source_string[i - 1], target_string[j - 1]))
                i -= 1
                j -= 1
            elif dp[i][j] == dp[i - 1][j] + 1:
                operations_performed.append(('DELETE', source_string[i - 1]))
                i -= 1
            else:
                operations_performed.append(('INSERT', target_string[j - 1]))
                j -= 1

    while (i != 0):
        operations_performed.append(('DELETE', source_string[i - 1]))
        i -= 1

    while (j != 0):
        operations_performed.append(('INSERT', target_string[j - 1]))
        j -= 1

    operations_performed.reverse()
    return [dp[m][n], operations_performed]

```

Sửa lỗi:

- Các vị trí trong source_string phải là i thay vì j, tương tự target_string là j thay vì i
- Sửa nếu dp[i][j] == dp[i – 1][j] + 1: thì đây là phép delete chứ không phải phép insert, còn sau dấu else là phép insert.
- Ở dưới while (i != 0) thì phải là phép delete, while(j != 0) thì là phép insert
- Ở phần return thì return về dp[m][n] (hay dp[len(source_string)][len(target_string)]

- Hàm __main__:

```python
if __name__ == "__main__":
    print("Enter the source string :")
    source_string = input().strip()
    print("Enter the target string :")
    target_string = input().strip()

    distance, operations_performed = find_minimum_edit_distance(source_string, target_string)

    insertions, deletions, replacements = 0, 0, 0
    for i in operations_performed:
        if i[0] == 'INSERT':
            insertions += 1
        elif i[0] == 'DELETE':
            deletions += 1
        else:
            replacements += 1

    print("Minimum edit distance : {}".format(distance))
    print("Number of insertions : {}".format(insertions))
    print("Number of deletions : {}".format(deletions))
    print("Number of replacements : {}".format(replacements))
    print("Total number of operations : {}".format(insertions + deletions + replacements))

    print("Actual Operations :")
    for i in range(len(operations_performed)):

        if operations_performed[i][0] == 'INSERT':
            print("{}) {} : {}".format(i + 1, operations_performed[i][0], operations_performed[i][1]))

        elif operations_performed[i][0] == 'DELETE':
            print("{}) {} : {}".format(i + 1, operations_performed[i][0], operations_performed[i][1]))

        else:
            print("{}) {} : {} by {}".format(i + 1, operations_performed[i][0], operations_performed[i][1], operations_performed[i][2]))

```

- Kết quả chạy chương trình với source_string là *INTENTION* và target_string là *EXECUTION*:

```
Enter the target string :
EXECUTION
Minimum edit distance : 8
Number of insertions : 1
Number of deletions : 1
Number of replacements : 3
Total number of operations : 5
Actual Operations :
1) DELETE : I
2) REPLACEMENT : N by E
3) REPLACEMENT : T by X
4) INSERT : C
5) REPLACEMENT : N by U
```

Cài đặt và thực thi mục 2:

- Hàm longest_common_subsequence để tìm số lượng dãy con chung không liên tiếp lớn nhất và tìm dãy con đó:

```python
def longest_common_subsequence(source_string, target_string):
    m = len(source_string)
    n = len(target_string)

    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1 , m + 1):
        dp[i][0] = 0

    for j in range(1 , n + 1):
        dp[0][j] = 0

    for i in range(1 , m + 1):
        for j in range(1 , n + 1):
            if source_string[i - 1] == target_string[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    lcss = []

    i = len(source_string)
    j = len(target_string)

    while(i != 0 and j != 0):
        if source_string[i - 1] == target_string[j - 1]:
            lcss.append(source_string[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1

    lcss.reverse()

    return [dp[m][n], lcss]

```

- Hàm __main__:

```python
if __name__ == "__main__":
    print("Enter the source string :")
    source_string = input().strip()
    print("Enter the target string :")
    target_string = input().strip()

    ans ,lcss = longest_common_subsequence(source_string, target_string)

    print("The number of longest common subsequence is :", ans)
    print("Longest common subsequence: ", lcss)
```

- Kết quả chạy chương trình với source_string là *ACADB* và *CBDA*:

```
Enter the source string :
ACADB
Enter the target string :
CBDA
The number of longest common subsequence is : 2
Longest common subsequence:  ['C', 'A']
```

Cài đặt và thực thi mục 3:

- Hàm dynamic_time_warping để tìm chuỗi đường đi wraping, phần tạo mảng và tính DTW:

```python
def dynamic_time_warping(A, B):
    m = len(A)
    n = len(B)

    DTW = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1 , m + 1):
        DTW[1][i] = abs(A[0] - B[i - 1]) + DTW[1][i - 1]

    for j in range(1 , n + 1):
        DTW[j][1] = abs(A[j - 1] - B[0]) + DTW[j - 1][1]

    for i in range(2 , m + 1):
        for j in range(2 , n + 1):
            DTW[i][j] = abs(A[i - 1] - B[j - 1]) + min(DTW[i - 1][j], DTW[i][j - 1], DTW[i - 1][j - 1])
```

- Phần tìm đường đi wraping:

```python
    wraping_road = []

    i = len(A)
    j = len(B)

    while(i != 1 and j != 1):
        wraping_road.append(DTW[i][j])
        Min = min(DTW[i - 1][j], DTW[i][j - 1], DTW[i - 1][j - 1])
        if Min == DTW[i - 1][j]:
            i -= 1
        elif Min == DTW[i][j - 1]:
            j -= 1
        else:
            i -= 1
            j -= 1

    while(i != 1):
        wraping_road.append(DTW[i][1])
        i -= 1

    while(j != 1):
        wraping_road.append(DTW[1][j])
        j -= 1
    wraping_road.append(DTW[1][1])

    return wraping_road

```

- Hàm __main__:

```python
if __name__ == "__main__":
    A = [1, 7, 4, 8, 2, 9, 6, 5, 2, 0]
    B = [1, 2, 8, 5, 5, 1, 9, 4, 6, 5]

    print("Wraping road is :", dynamic_time_warping(A, B))
```

- Kết quả chạy chương trình với 2 chuỗi thời gian A và B như trên:

```
Wraping road is : [17, 12, 9, 9, 9, 7, 7, 6, 3, 2, 1, 0]
```